SAST — Static Application Security Testing

PyGoat — Vulnerable Web Application

AUTORE Charaf Mansori

PROGETTO PyGoat — applicazione Django volutamente vulnerabile

STRUMENTO SonarQube (+ scansione OSV delle dipendenze)

DATA ANALISI 21 Maggio 2026

DATA REPORT 04 Giugno 2026 (rev. 23 Ottobre 2026)

23	146	198	157	524
BLOCKER	HIGH	MEDIUM	LOW	TOTALE

LIVELLO DI RISCHIO: CRITICO

Report didattico: PyGoat è un'applicazione intenzionalmente vulnerabile, analizzata in laboratorio.

INDICE

1.	Executive Summary
2.	Acronimi e Definizioni
3.	Risultati Generali e Statistiche
4.	Grafici di Distribuzione
5.	Vulnerabilità Critiche (Blocker / High)
6.	Analisi delle Security Issues
7.	Scansione OSV delle dipendenze
8.	Best Practices e Remediation
9.	Conclusioni e Action Plan

1. Executive Summary

L'analisi di Static Application Security Testing (SAST) condotta con **SonarQube** sull'applicazione **PyGoat** ha rilevato **524 problematiche**, di cui **23 Blocker** (severità massima). L'analisi è stata eseguita il 21 Maggio 2026; il report è stato redatto il 4 Giugno 2026.

PyGoat è un'applicazione web Django/Python **intenzionalmente vulnerabile** (OWASP Top 10), usata a scopo didattico. Il livello di rischio «CRITICO» descrive quindi cosa accadrebbe se un'applicazione con queste caratteristiche venisse esposta in un ambiente reale: andrebbe esclusa da qualunque deployment di produzione senza una remediation completa.

In allegato è disponibile la scansione OSV (Open Source Vulnerability) delle dipendenze (`OSV_Scan_PyGoat.html`), che complementa il SAST individuando le vulnerabilità note (CVE) nei pacchetti di terze parti (sezione 7).

Criticità principali

Vulnerabilità	CWE	Impatto
Credenziali hardcoded — Django SECRET_KEY e credenziali DB nel codice	CWE-798	Compromissione di sessioni e dati
Chiavi segrete esposte — API key, token JWT e segreti crittografici nel codice	CWE-798	Violazione della separazione dei segreti
Configurazioni inadeguate — DEBUG=True, ALLOWED_HOSTS=*, CSRF disabilitata	CWE-16 / CWE-352	Esposizione di informazioni, attacchi CSRF
Autenticazione carente — policy deboli, nessuna protezione brute force	CWE-287	Bypass autenticazione, account takeover
SQL Injection — query costruite per concatenazione di stringhe	CWE-89	Esposizione del database
Cross-Site Scripting — input non sanitizzato nei template	CWE-79	JavaScript arbitrario, furto di sessione

2. Acronimi e Definizioni

Sigla	Significato	Descrizione
SAST	Static Application Security Testing	Analisi del codice sorgente senza eseguirlo, alla ricerca di pattern vulnerabili. Individua i problemi presto e si automatizza in CI/CD.
DAST	Dynamic Application Security Testing	Test dell'applicazione in esecuzione, dal punto di vista di un attaccante; complementare al SAST e non richiede il codice.
CI/CD	Continuous Integration / Delivery	Automazione di build, test e rilascio. Una pipeline sicura include SAST, controllo delle dipendenze e quality gate prima del deploy.
OWASP	Open Worldwide Application Security Project	Organizzazione no-profit che produce linee guida e strumenti per la sicurezza applicativa (Top 10, Testing Guide, ZAP).
CWE	Common Weakness Enumeration	Catalogo delle debolezze software, mantenuto da MITRE. Ogni debolezza ha un ID (es. CWE-89 SQL Injection, CWE-79 XSS).
CVE	Common Vulnerabilities and Exposures	Identificatore univoco di una vulnerabilità nota e pubblica.
CVSS	Common Vulnerability Scoring System	Sistema di punteggio della gravità (v3.x): Critical ≥ 9.0, High 7.0–8.9, Medium 4.0–6.9, Low 0.1–3.9.
OSV	Open Source Vulnerability	Database aperto di vulnerabilità per gli ecosistemi open source, promosso da Google; lo scanner confronta le dipendenze del progetto con le vulnerabilità note.

3. Risultati Generali e Statistiche

La scansione SonarQube ha prodotto **524 issue**, classificate sia per categoria sia per severità (i due totali coincidono).

3.1 Distribuzione per categoria

Categoria	Issue	%	Priorità remediation
Security	50	9.5%	MASSIMA
Reliability	97	18.5%	ALTA
Maintainability	377	72.0%	CONTINUA
Totale	**524**	**100%**	—

3.2 Distribuzione per severità

Severità	Issue	%	Timeline remediation	Priorità
BLOCKER	23	4.4%	24 – 48 ore	CRITICA
HIGH	146	27.9%	1 – 2 settimane	URGENTE
MEDIUM	198	37.8%	1 – 3 mesi	IMPORTANTE
LOW	157	30.0%	Continuativo	MEDIA
Totale	**524**	**100%**	—	—

4. Grafici di Distribuzione

4.1 Issue per categoria

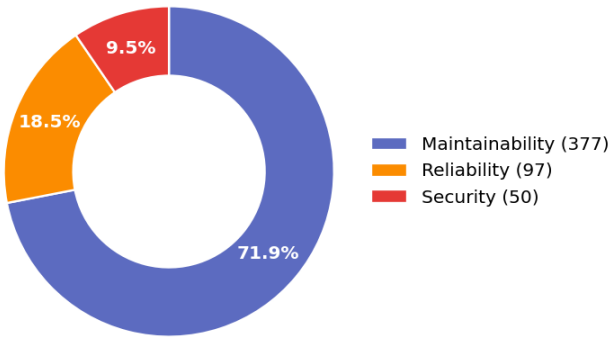


Figura 1 — Il 72% delle issue riguarda la Maintainability (code smell, duplicazioni, leggibilità); Security (9.5%) e Reliability (18.5%) richiedono gli interventi più urgenti.

4.2 Issue per severità

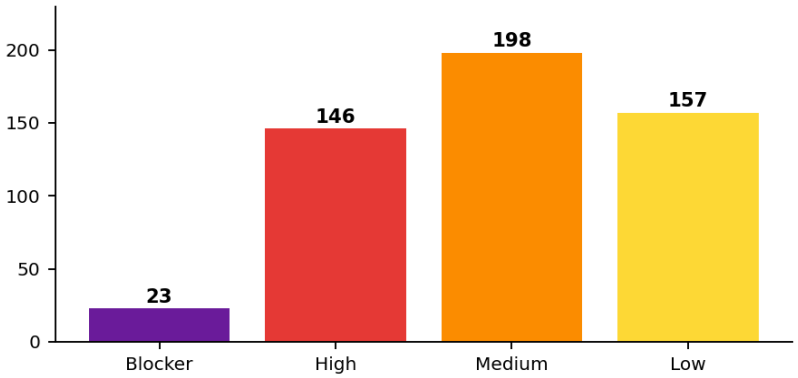


Figura 2 — Le 23 issue Blocker impediscono il deployment; le 146 High vanno risolte entro 2 settimane per portare il rischio a un livello accettabile.

4.3 Security Issues per categoria OWASP Top 10

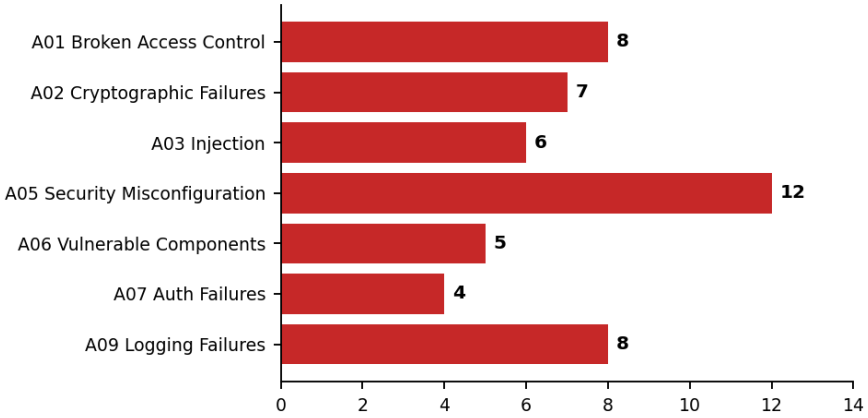


Figura 3 — Distribuzione delle 50 Security Issues: A05 Security Misconfiguration è la categoria più frequente (12), seguita da A01 e A09 (8 ciascuna).

5. Vulnerabilità Critiche (Blocker / High)

Per ciascun finding: identificativo, severità, componente, codice vulnerabile, codice corretto e remediation.

SEC-001 — Django SECRET_KEY hardcoded BLOCKER

CWE: *CWE-798* **Componente:** *pygoat/settings.py* **OWASP:** *A02 – Cryptographic Failures*

La SECRET_KEY di Django è scritta nel file di configurazione. Questa chiave firma i cookie di sessione e i token CSRF: se compromessa, un attaccante può falsificare sessioni e impersonare qualsiasi utente, amministratori inclusi.

CODICE VULNERABILE

```
# pygoat/settings.py
SECRET_KEY = 'hardcoded_secret_key_12345'   # VULNERABILE
```

CODICE CORRETTO

```
import os
from django.core.exceptions import ImproperlyConfigured

SECRET_KEY = os.environ.get('DJANGO_SECRET_KEY')
if not SECRET_KEY:
    raise ImproperlyConfigured('DJANGO_SECRET_KEY non impostata')
```

Remediation: rimuovere la chiave dal codice, generarne una nuova (`django.core.management.utils.get_random_secret_key()`) e conservarla in un secret manager. Considerare compromesse tutte le chiavi già esposte.

SEC-002 — Credenziali del database hardcoded BLOCKER

CWE: *CWE-798* **Componente:** *pygoat/settings.py* **OWASP:** *A02 – Cryptographic Failures*

Utente e password del database sono scritti nel sorgente. Chi accede al codice può leggere, modificare o cancellare i dati e usare il database per muoversi lateralmente.

CODICE VULNERABILE

```
DATABASES = {
  'default': {
    'ENGINE': 'django.db.backends.postgresql',
    'NAME': 'pygoat',
    'USER': 'admin',          # VULNERABILE
    'PASSWORD': 'admin123',   # VULNERABILE
    'HOST': '192.168.1.10',
  }
}
```

CODICE CORRETTO

```
import os
DATABASES = {
  'default': {
    'ENGINE': 'django.db.backends.postgresql',
    'NAME': os.environ['DB_NAME'],
    'USER': os.environ['DB_USER'],
    'PASSWORD': os.environ['DB_PASSWORD'],
    'HOST': os.environ.get('DB_HOST', 'localhost'),
  }
}
```

Remediation: usare variabili d'ambiente o un secret manager (es. django-environ, python-decouple, Vault).

SEC-003 — DEBUG abilitato e ALLOWED_HOSTS aperto — BLOCKER

CWE: *CWE-215 / CWE-200* **Componente:** *pygoat/settings.py* **OWASP:** *A05 – Security Misconfiguration*

Con DEBUG=True Django mostra stack trace con variabili locali, percorsi del filesystem e configurazioni; ALLOWED_HOSTS=['*'] accetta qualsiasi host. Informazioni preziose per un attaccante.

CODICE VULNERABILE

```
DEBUG = True                # VULNERABILE
ALLOWED_HOSTS = ['*']       # VULNERABILE
```

CODICE CORRETTO

```
import os
DEBUG = os.environ.get('DJANGO_DEBUG', 'False') == 'True'
ALLOWED_HOSTS = os.environ.get('DJANGO_ALLOWED_HOSTS', '').split(',')
```

Remediation: disattivare DEBUG fuori dallo sviluppo e dichiarare esplicitamente gli host ammessi.

SEC-004 — SQL Injection per concatenazione di stringhe — HIGH

CWE: *CWE-89* **Componente:** *introduction/views.py* **OWASP:** *A03 – Injection*

Le query SQL sono costruite concatenando l'input dell'utente, senza query parametrizzate. Un attaccante può alterare la logica SQL per estrarre dati o aggirare l'autenticazione.

CODICE VULNERABILE

```
username = request.POST.get('username')
query = "SELECT * FROM users WHERE username='" + username + "'"
cursor.execute(query)   # VULNERABILE
```

CODICE CORRETTO

```
from django.contrib.auth.models import User

# Metodo 1: ORM (consigliato)
user = User.objects.filter(username=username).first()

# Metodo 2: query parametrizzata
cursor.execute('SELECT * FROM users WHERE username=%s', [username])
```

Remediation: usare sempre l'ORM o query parametrizzate; mai concatenare input utente.

SEC-005 — Cross-Site Scripting (XSS) — HIGH

CWE: *CWE-79* **Componente:** *templates/*.html* **OWASP:** *A03 – Injection*

L'input utente viene mostrato nei template disattivando l'escaping automatico di Django (filtro `|safe` o `autoescape off`). Consente di iniettare JavaScript: furto di cookie di sessione, redirect malevoli, phishing.

CODICE VULNERABILE

```
{{ user_input|safe }}
{% autoescape off %}{{ user_comment }}{% endautoescape %}
```

CODICE CORRETTO

```
{# Django applica l'escaping di default #}
{{ user_input }}

# Se serve HTML, sanificare lato server con bleach:
import bleach
clean = bleach.clean(user_input, tags=['b', 'i', 'p'])
```

Remediation: mantenere l'autoescape attivo; sanificare con una allowlist quando l'HTML è davvero necessario; aggiungere una Content-Security-Policy.

6. Analisi delle Security Issues

Riepilogo delle 50 Security Issues, con CWE, severità, componente e remediation proposta.

ID	Categoria / CWE	Severità	Componente	Remediation
SEC-001	Hardcoded Secret Key / CWE-798	BLOCKER	pygoat/settings.py	Env Variable
SEC-002	Hardcoded DB Credentials / CWE-798	BLOCKER	pygoat/settings.py	Secret Manager
SEC-003	Debug Mode Enabled / CWE-215	BLOCKER	pygoat/settings.py	Disable DEBUG
SEC-004	SQL Injection / CWE-89	HIGH	introduction/views.py	Parametrize
SEC-005	XSS Reflected / CWE-79	HIGH	templates/*.html	Escape Output
SEC-006	Insecure Deserialization / CWE-502	BLOCKER	lab/views.py	Safe Deserialize
SEC-007	Path Traversal / CWE-22	BLOCKER	introduction/views.py	Sanitize Path
SEC-008	Command Injection / CWE-78	BLOCKER	lab/admin_views.py	Avoid exec()
SEC-009	Weak Password Policy / CWE-521	HIGH	user/validators.py	Enforce Policy
SEC-010	Missing Auth Check / CWE-862	HIGH	introduction/views.py	@login_required
SEC-011	CSRF Disabled / CWE-352	BLOCKER	pygoat/settings.py	Enable CSRF
SEC-012	Broken Access Control / CWE-284	HIGH	admin/views.py	RBAC
SEC-013	Sensitive Data in Logs / CWE-532	HIGH	logs/error.log	Mask PII
SEC-014	Insecure Cookie / CWE-614	HIGH	pygoat/settings.py	Set Secure Flag
SEC-015	Missing HSTS / CWE-319	MEDIUM	pygoat/settings.py	Add HSTS Header
SEC-016	Open Redirect / CWE-601	HIGH	user/views.py	Whitelist URLs
SEC-017	File Upload Unrestricted / CWE-434	HIGH	upload/views.py	Validate Type
SEC-018	Hardcoded API Key / CWE-798	BLOCKER	integration/client.py	Env Variable
SEC-019	XML External Entity / CWE-611	HIGH	xml_parser/views.py	Disable XXE
SEC-020	SSRF / CWE-918	HIGH	fetch/views.py	Allowlist URLs
SEC-021	Weak Crypto Algorithm / CWE-327	HIGH	crypto/utils.py	Use SHA-256+
SEC-022	Cleartext Password Storage / CWE-256	BLOCKER	user/models.py	Bcrypt/Argon2
SEC-023	Session Fixation / CWE-384	HIGH	auth/views.py	Regenerate ID
SEC-024	Missing Input Validation / CWE-20	MEDIUM	forms/*.py	Add Validators
SEC-025	Verbose Error Messages / CWE-209	MEDIUM	pygoat/settings.py	Generic Errors
SEC-026	Unvalidated Redirect / CWE-601	MEDIUM	user/views.py	Safe Redirect
SEC-027	Insufficient Logging / CWE-778	MEDIUM	pygoat/settings.py	Enable Logging
SEC-028	Broken Object Auth / CWE-639	HIGH	api/views.py	Check Ownership
SEC-029	Mass Assignment / CWE-915	HIGH	user/serializers.py	Whitelist Fields
SEC-030	XXE in Upload / CWE-611	HIGH	upload/xml_views.py	Disable DTD
SEC-031	Prototype Pollution / CWE-1321	MEDIUM	static/js/utils.js	Sanitize
SEC-032	Missing CSP Header / CWE-693	MEDIUM	middleware/	Add CSP
SEC-033	Clickjacking / CWE-1021	MEDIUM	pygoat/settings.py	X-Frame-Options
SEC-034	Template Injection / CWE-1336	BLOCKER	templates/render.py	Sandbox
SEC-035	Race Condition / CWE-362	HIGH	transaction/views.py	Atomic Ops
SEC-036	Insecure Random / CWE-338	HIGH	token/utils.py	secrets module

ID	Categoria / CWE	Severità	Componente	Remediation
SEC-037	LDAP Injection / CWE-90	HIGH	auth/ldap_views.py	Escape Input
SEC-038	XPath Injection / CWE-91	MEDIUM	xml/xpath_views.py	Parameterize
SEC-039	Missing Rate Limiting / CWE-307	MEDIUM	auth/views.py	Throttle
SEC-040	Hardcoded JWT Secret / CWE-798	BLOCKER	api/auth.py	Env Variable
SEC-041	JWT Algorithm None / CWE-347	BLOCKER	api/auth.py	Enforce RS256
SEC-042	Weak Session Token / CWE-330	HIGH	session/utils.py	Secure Random
SEC-043	Cleartext Transmission / CWE-319	HIGH	api/client.py	Enforce HTTPS
SEC-044	Cache Poisoning / CWE-444	MEDIUM	cache/middleware.py	Validate Keys
SEC-045	Dependency Vulnerability / CWE-1104	HIGH	requirements.txt	Update Deps
SEC-046	Missing Security Headers / CWE-693	MEDIUM	pygoat/settings.py	Add Headers
SEC-047	Business Logic Flaw / CWE-840	MEDIUM	shop/views.py	Review Logic
SEC-048	Insecure File Permissions / CWE-732	LOW	scripts/backup.py	chmod 600
SEC-049	Information Disclosure / CWE-200	LOW	api/views.py	Filter Output
SEC-050	Missing Integrity Check / CWE-345	LOW	update/views.py	Verify Hash

7. Scansione OSV delle dipendenze

Oltre al SAST sul codice, è stata eseguita una scansione OSV dei file `requirements.txt` di PyGoat e dei suoi laboratori dockerizzati. Il file completo è allegato: `OSV_Scan_PyGoat.html` (da aprire nel browser).

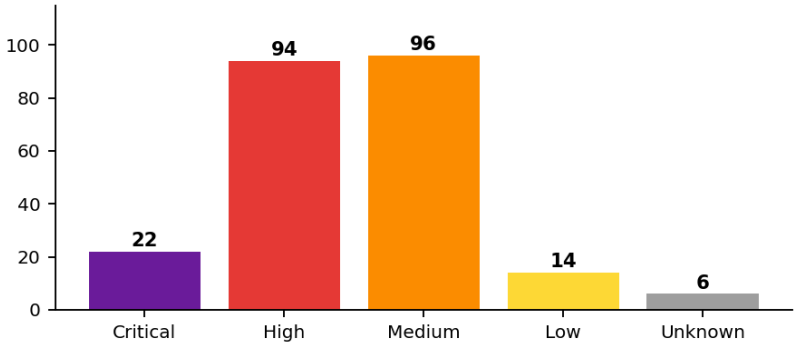


Figura 4 — Totali riportati dallo scanner: 232 vulnerabilità (22 Critical, 94 High, 96 Medium, 14 Low, 6 Unknown).

Nota metodologica. Lo scanner ha letto ciascun file di dipendenze in due modalità («lockfile» e «unknown»), quindi alcuni pacchetti sono contati due volte. Eliminando i duplicati (stesso file, pacchetto e versione) i finding unici sono circa **145**: 12 Critical, 59 High, 62 Medium, 7 Low, 5 Unknown. Il dato dello scanner (232) resta comunque il riferimento del file allegato.

Pacchetti più esposti (requirements.txt principale)

Pacchetto	Versione	Critical	High	Medium	Low	Unknown	Totale
Django	4.2	6	18	16	3	0	**43**
cryptography	39.0.1	0	4	2	0	4	10
Werkzeug	2.1.2	0	4	4	1	0	9
urllib3	1.26.9	0	5	3	0	0	8
Pillow	9.4.0	1	3	2	0	1	7
PyJWT	2.4.0	0	3	1	1	0	5
Requests	2.28.2	0	0	4	0	0	4
PyYAML	5.1	3	0	0	0	0	3
django-allauth	0.52.0	0	0	3	0	0	3

Remediation: aggiornare Django alla versione LTS corrente, aggiornare le librerie crittografiche e di rete (cryptography, urllib3, Werkzeug, Pillow, PyJWT, PyYAML) e inserire `pip-audit` o OSV-Scanner nella pipeline CI/CD.

8. Best Practices e Remediation

Pratica	Descrizione	Strumenti	Priorità
Secret management	Non inserire credenziali, chiavi API o token nel codice: variabili d'ambiente, vault o secret manager.	HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, python-decouple	CRITICA
Input validation	Validare e sanificare ogni input, lato server e lato client.	Django validators, bleach, pydantic	ALTA
Controlli OWASP Top 10	Implementare e testare i controlli per ciascuna categoria.	OWASP ZAP, ASVS, Cheat Sheet Series	ALTA
SAST in CI/CD	Scansione automatica che blocca merge/deploy con vulnerabilità critiche.	SonarQube, Bandit, Semgrep	ALTA
Dependency scanning	Controllo periodico delle dipendenze e aggiornamento di requirements.txt.	pip-audit, OSV-Scanner, OWASP Dependency-Check	ALTA
Code review	Revisione obbligatoria da un'altra persona prima del merge (principio dei quattro occhi).	GitHub PR reviews, GitLab MR	MEDIA
Security testing	DAST e penetration test periodici o a ogni release maggiore.	OWASP ZAP, Burp Suite, Nessus	MEDIA
Logging & monitoring	Logging centralizzato degli eventi di sicurezza con alert.	ELK Stack, Graylog, SIEM	MEDIA
Security training	Formazione continua su secure coding e OWASP Top 10.	OWASP, SANS	PIANIFICATA

9. Conclusioni e Action Plan

9.1 Valutazione complessiva

PyGoat presenta un livello di rischio **critico**: 23 issue Blocker (tra cui credenziali hardcoded, Command Injection, Path Traversal, Template Injection e deserializzazione insicura), 146 High e numerose vulnerabilità OWASP Top 10 volutamente presenti. Trattandosi di un'applicazione didattica, ciò è atteso; l'analisi mostra però come un solo difetto Blocker, come una SECRET_KEY hardcoded, basti a compromettere l'intera sicurezza applicativa.

9.2 Azioni immediate (se fosse un'applicazione reale)

#	Azione	Dettaglio
1	**Bloccare il deployment**	Nessun rilascio finché le 23 Blocker non sono risolte; quality gate in CI/CD che blocchi il deploy.
2	**Remediation urgente (0–48 h)**	Rimuovere credenziali, chiavi API e segreti dal codice e rigenerare quelli esposti.
3	**Secret manager (0–48 h)**	Gestione centralizzata dei segreti, integrata nel deployment.
4	**Fix High (1–2 settimane)**	Priorità a SQL Injection, XSS, CSRF e autenticazione; poi nuova scansione SAST.
5	**SAST in CI/CD (settimana 2)**	SonarQube come quality gate obbligatorio.
6	**Security training (mese 1)**	Formazione su secure coding, OWASP Top 10 e best practice Django/Python.

9.3 Roadmap di remediation

Fase	Periodo	Attività	Obiettivo
CRITICA	Settimana 1	Remediation Blocker, secret manager, rigenerazione chiavi, disattivare DEBUG	23/23 Blocker risolte
URGENTE	Sett. 2–4	Fix High, SAST in CI/CD, dependency scanning	High ridotte a < 20

Fase	Periodo	Attività	Obiettivo
IMPORTANTE	Mesi 2–3	Fix Medium, training, penetration test completo, code review	Medium ridotte a < 50
CONTINUA	Mesi 4–12	Monitoraggio, scansioni SAST periodiche, miglioramento incrementale	Rischio BASSO, Quality Gate A